ChatGPT

Exploratory Data Analysis & Feature Engineering for Insurance Renewal

This notebook performs a detailed EDA on the insurance renewal dataset, examining each feature's distribution and its relationship with the binary target `renewal`. We compute descriptive statistics, visualize patterns, and engineer new features. Finally, we assess class imbalance and prepare a plan for modeling (logistic regression, XGBoost, neural nets, etc.). The code is clean and well-commented, with rationale and findings explained throughout.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load data
df = pd.read_csv('train_ZoGVYWq.txt')
print(df.shape)
df.head(5)
```

The dataset has **79,853 rows** and columns:

```
['id', 'perc_premium_paid_by_cash_credit', 'age_in_days', 'Income',
 'Count_3-6_months_late', 'Count_6-12_months_late',
 'Count_more_than_12_months_late', 'application_underwriting_score',
 'no_of_premiums_paid', 'sourcing_channel', 'residence_area_type',
 'premium', 'renewal']
```

We note some missing values (empty fields) and decide how to handle them:

```
# Check missing values
print(df.isnull().sum())
```

- **Late-payment counts** (`Count_..._late`) are missing (NaN) for 97 rows; these occur when the customer has very few payments (e.g. 2 or 3), so we **fill them with 0** (no late payments) for analysis. - **Underwriting score** has 2,974 missing values. We will **drop those rows** (≃3.7% of data) since imputing this score (nearly always near 99) would have little impact.

```
df['Count_3-6_months_late'].fillna(0, inplace=True)
df['Count_6-12_months_late'].fillna(0, inplace=True)
df['Count_more_than_12_months_late'].fillna(0, inplace=True)
df_clean = df.dropna(subset=['application_underwriting_score']).copy()
```

```
print(df_clean.shape, "rows remaining after dropping missing underwriting
scores")
```

After cleaning, we have **76,879 rows**. The **target distribution** is highly imbalanced:

```
print(df_clean['renewal'].value_counts(normalize=True))
```

```
1    0.938
0    0.062
```

Only about **6.2%** of cases are *non-renewals* (`renewal=0`). We will need to address this imbalance later (e.g. SMOTE/ADASYN, class weights).

Per-Feature Analysis

Below we examine each feature's distribution and its relationship with the target `renewal`. We report key statistics (mean, median, etc.) and group statistics for the two target classes.

`perc_premium_paid_by_cash_credit` (continuous 0–1)

Fraction of premium paid by cash/credit card. Distribution is *skewed low*: median ≈0.174 (17%) and 75%-ile ≈0.535. Only ~6.3% of values are exactly 0.

```
print(df_clean['perc_premium_paid_by_cash_credit'].describe().round(3))
# Compare by renewal
stats = df_clean.groupby('renewal')
['perc_premium_paid_by_cash_credit'].describe()
print(stats.round(3))
```

```
count      76879.000
mean           0.315
std            0.330
min            0.000
25%            0.038
50%            0.174
75%            0.535
max            1.000
Name: perc_premium_paid_by_cash_credit, dtype: float64

               count   mean    std    min    25%    50%    75%   max
renewal
0             4798.0  0.619  0.350  0.000  0.312  0.711  0.960  1.000
1            72081.0  0.295  0.319  0.000  0.035  0.155  0.488  1.000
```

Interpretation: Customers who **did not renew** (`renewal=0`) paid a much larger fraction of their premium by cash/credit on average (~0.62) than those who **did renew** (~0.30). In other words, a high *perc_premium_paid_by_cash_credit* is strongly associated with non-renewal. This suggests it is a valuable predictor.

`age_in_days` (converted to years)

We interpret `age_in_days` as customer age. We convert to years for readability:

```
df_clean['age_years'] = df_clean['age_in_days'] / 365.25
print(df_clean['age_years'].describe().round(2))
```

```
count    76879.00
mean        51.47
std         13.97
min         21.00
25%         40.99
50%         50.99
75%         61.00
max        101.96
Name: age_years, dtype: float64
```

Ages range roughly from 21 to 102 years, with median ~51. Comparing by renewal:

```
print(df_clean.groupby('renewal')['age_years'].describe().round(2))
```

```
           count   mean    std   min    25%    50%    75%    max
renewal
0         4798.0  46.45  12.65  21.00  36.99  45.99  54.00  90.97
1        72081.0  51.81  13.99  21.00  41.01  51.00  61.98 101.96
```

Interpretation: Non-renewers (`0`) are noticeably **younger** on average (~46.5 yrs) than renewers (~51.8 yrs). The renewal rate declines for younger customers (who are less likely to renew), while older customers tend to renew. This could reflect different risk or policy lapse behavior by age.

`Income` (numeric)

Annual income (presumably in currency units). It is skewed right with a huge maximum (~53M!), suggesting a few outliers or very high incomes. Typical quartiles:

```
count    76879.00
mean    208797.30
std     369696.93
min      24030.00
25%     109660.00
50%     168240.00
```

```
75%    254285.00
max  53821900.00
```

Group by renewal:

```
print(df_clean.groupby('renewal')['Income'].describe().round(0))
```

```
           count    mean     std     min     25%      50%     75%       max
renewal
0         4798.0  179664  210892  24030.0   91475  141040.0  212770   7500070
1        72081.0  210736  377827  24030.0  111360  171110.0  255140  53821900
```

Interpretation: Those who **renew** have higher incomes on average (~210k) than non-renewers (~180k). This suggests higher-income customers are more likely to renew their insurance. (The effect is present but the variable has outliers; a log or robust transformation might be used later.)

Late-payment Counts (`Count_3-6_months_late`, `Count_6-12_months_late`, `Count_more_than_12_months_late`)

These three integer features count how many payments were late by 3–6, 6–12, or >12 months. Most customers have **zero** late payments (as seen by the very high 25% and 50% = 0 in `describe()`). For brevity, we focus on a combined metric:

```
df_clean['total_late_counts'] = (
    df_clean['Count_3-6_months_late'] +
    df_clean['Count_6-12_months_late'] +
    df_clean['Count_more_than_12_months_late']
)
print(df_clean['total_late_counts'].value_counts().head(5))
print(df_clean['total_late_counts'].describe().round(2))
print(df_clean.groupby('renewal')['total_late_counts'].describe().round(2))
```

```
0.0    61074
1.0     9334
2.0     3124
3.0     1453
4.0      791
...
count   76879.00
mean        0.43
std         1.35
min         0.00
25%         0.00
50%         0.00
75%         0.00
max        19.00
```

```
            count  mean   std  min   25%   50%   75%   max
renewal
0          4798.0  1.85  2.25  0.0  0.00  1.00  3.00  18.0
1         72081.0  0.30  0.83  0.0  0.00  0.00  0.00  19.0
```

Interpretation: About 79% of customers have **no late payments** at all (total_late=0). The average number of late payments among *non-renewers* is **much higher** (mean ≈1.85) than among renewers (mean ≈0.30). In fact, the renewal rate **drops sharply** as total late counts increase. We confirm with a simple check:

```
# Binary feature: any late payment
df_clean['any_late'] = (df_clean['total_late_counts'] > 0).astype(int)
print(df_clean.groupby('any_late')['renewal'].mean())
```

```
any_late
0    0.9714
1    0.8068
Name: renewal, dtype: float64
```

Customers with **no late payments** renew ~97.1% of the time, whereas those with *any* late payment renew only ~80.7%. **Late payments are by far the strongest predictors of non-renewal.**

We will use `total_late_counts` (and possibly the rate of late-per-premium) as engineered features. For example:

```
df_clean['late_rate'] = df_clean['total_late_counts'] /
df_clean['no_of_premiums_paid']
```

where many customers have `late_rate=0`.

`application_underwriting_score`

Score (approximately 0–100) assessing the application risk. We see it is **very high (mean ≈99.07)** and tightly distributed (std ≈0.74). Most values are in [98, 100]. Group stats:

```
print(df_clean.groupby('renewal')
['application_underwriting_score'].describe().round(2))
```

```
            count   mean   std   min    25%    50%    75%    max
renewal
0          4798.0  98.87  0.88  92.24  98.53  99.05  99.48  99.89
1         72081.0  99.08  0.73  91.90  98.83  99.22  99.54  99.89
```

Interpretation: Non-renewers have a slightly lower average score (~98.87 vs 99.08), but this difference is very small compared to the tight range of scores. Almost all scores are ≥91.9, so this feature may have only weak predictive power.

`no_of_premiums_paid`

Number of premiums the customer has paid (integer). Summary:

```
print(df_clean['no_of_premiums_paid'].describe())
print(df_clean.groupby('renewal')['no_of_premiums_paid'].mean())
```

Mean ~11.05, median 10, range 2–59. Renewers have a slightly higher mean (≈11.08) than non-renewers (≈10.66). This suggests long-term customers (who've paid more premiums) may be slightly more likely to renew, but the effect is small.

`premium`

Annual premium amount. Summaries:

```
print(df_clean['premium'].describe().round(0))
print(df_clean.groupby('renewal')['premium'].describe().round(0))
```

```
count    76879.00
mean     11006.06
std       9418.41
min       1200.00
25%       5400.00
50%       7500.00
75%      13800.00
max      60000.00

           count    mean    std   min    25%    50%    75%    max
renewal
0         4798.0  9721.0  8707  1200.0  5400.0  7500.0 11700.0 60000.0
1        72081.0 11092.0  9458  1200.0  5400.0  7500.0 13800.0 60000.0
```

Interpretation: Customers who renewed paid *higher premiums* on average (~11092 vs 9721). Higher-premium contracts seem slightly more likely to renew.

Categorical Features

`sourcing_channel` **(A–E)**

Counts by channel:

```
print(df_clean['sourcing_channel'].value_counts())
print(df_clean.groupby('sourcing_channel')['renewal'].mean().round(3))
```

```
A    40926
B    16076
C    11844
D     7434
E      599

sourcing_channel
A    0.9456
B    0.9361
C    0.9260
D    0.9163
E    0.9249
Name: renewal, dtype: float64
```

Interpretation: Channel A (most common) has the highest renewal rate (~94.6%), while channel D has the lowest (~91.6%). The effect is modest but suggests `sourcing_channel` carries information. We will encode these as dummy variables for modeling.

`residence_area_type` **(Urban/Rural)**

Counts and renewal:

```
print(df_clean['residence_area_type'].value_counts())
print(df_clean.groupby('residence_area_type')['renewal'].mean())
```

```
Urban    46343
Rural    30536

residence_area_type
Rural    0.9371
Urban    0.9379
Name: renewal, dtype: float64
```

Interpretation: Urban vs Rural customers renew at almost the same rate (~93.7%). This feature appears **neutral**; it likely adds little predictive power. Nevertheless, we can include it via encoding if desired.

Feature Summary

From the above, the **strongest predictors** are clearly the late-payment features and `perc_premium_paid_by_cash_credit`. Moderate signals come from `age`, `Income`, `premium`, and `sourcing_channel`. Weak predictors include underwriting score and area type. We will engineer

features combining the three late-payment columns into `total_late_counts` and `late_rate`, and also consider a binary `any_late`.

```
# Example: mutual information to quantify feature importance
from sklearn.feature_selection import mutual_info_classif
from sklearn.preprocessing import LabelEncoder

# Prepare X, y for MI
X = df_clean.drop(['id','renewal'], axis=1).copy()
# Encode categoricals as integers for MI
for col in ['sourcing_channel','residence_area_type']:
    X[col] = LabelEncoder().fit_transform(X[col])
X = X.fillna(X.mean())
y = df_clean['renewal']
mi = mutual_info_classif(X, y, random_state=0)
mi_scores = sorted(zip(X.columns, mi), key=lambda x: x[1], reverse=True)
print("Top features by mutual information:")
for feat, score in mi_scores[:5]:
    print(f"{feat}: {score:.3f}")
```

```
Top features by mutual information:
late_rate: 0.041
total_late_counts: 0.039
Count_6-12_months_late: 0.026
perc_premium_paid_by_cash_credit: 0.026
Count_3-6_months_late: 0.022
...
```

Interpretation: This confirms our findings: `late_rate` (ratio of late payments) and `total_late_counts` carry the most information about renewal. The cash/credit percentage and individual late counts also rank highly. Many other features (income, premiums, etc.) have much lower MI. Thus, the EDA suggests we should focus on these high-impact features in modeling.

Class Imbalance Handling

The target `renewal` is very imbalanced (94% renew vs 6% not). As Brownlee (2021) notes, *"most machine learning techniques will ignore the minority class, although it is typically most important"* [1]. We must address this imbalance to improve prediction of the rare `renewal=0` cases.

Common strategies include:

- **Oversampling the minority class:** SMOTE (Synthetic Minority Oversampling Technique) is widely used [2]. SMOTE synthetically generates new minority-class examples by interpolating between existing ones. For example, it picks a minority sample and a nearest neighbor, then creates a new sample along the line between them. ADASYN (Adaptive Synthetic Sampling) is a related method that focuses on "hard" minority examples [3]: it generates more synthetic points

for minority instances that are difficult to learn (near the decision boundary). We will experiment with SMOTE and ADASYN to balance the training set.

- **Class-weighted algorithms:** Many classifiers (Logistic Regression, SVM, XGBoost, etc.) allow weighting classes so that mistakes on minority samples are penalized more. For example, setting `class_weight='balanced'` in scikit-learn weights classes inversely to their frequency.

- **Ensemble methods for imbalance:** Methods like BalancedBagging, BalancedRandomForest, or EasyEnsemble combine sampling with bagging/boosting to handle skew [4]. Such techniques could be applied if needed.

- **Undersampling the majority:** Randomly removing some renewal=1 cases can balance classes, but risks throwing away data. Often better to oversample minority (or a combination of both).

We will evaluate these options (e.g. by cross-validation performance) before final modeling. Visual diagnostics (not shown) like the above class-distribution bar chart help motivate these choices.

Visualizations (Distributions & Relationships)

We generate several plots to visualize feature distributions overall and by `renewal`. For example:

- **Histograms/Kernel Density:** Show how numeric features differ by class. E.g. a histogram of `age_years` for renewers vs non-renewers, or `premium` for each class.
- **Boxplots:** Compare medians and interquartile ranges of features by renewal outcome (e.g. boxplot of `Income` split by `renewal`).
- **Bar charts:** Categorical vs target (e.g. bar chart of renewal rate for each `sourcing_channel`).

(Plots not shown here, but would be included in the notebook.) These visual checks confirm the trends seen in the tables above: e.g., non-renewal is concentrated among younger ages, higher perc_paid_by_cash, and any late payments.

```
# Example: boxplot of income by renewal
sns.boxplot(data=df_clean, x='renewal', y='Income')
plt.title('Income by Renewal (0=No, 1=Yes)')
plt.show()
```

Preparation for Modeling

Based on the EDA, we will select and preprocess features as follows:

- **Feature selection:** We include the most informative features:
- **Late-payment features:** `total_late_counts` and/or `late_rate` (and possibly the three individual counts if needed). We may also use the binary `any_late`.
- **perc_premium_paid_by_cash_credit:** Strong signal from EDA.
- **Age:** `age_years` (or `age_in_days`).
- **Income.**

- **Premium.**
- **No_of_premiums_paid.**
- **Sourcing channel & residence area:** encoded (e.g. one-hot encoding A/B/C/D/E and Urban/Rural).
- **Underwriting score:** Possibly included, though it had weak effect. We drop the `id` column (not predictive).
- **Feature engineering:** We have already created:
- `total_late_counts`, `late_rate`, `any_late`.
- We attempted ratios (like `premium_to_income`) but saw no added benefit, so we omit them.
- **Encoding:** Convert categorical channels (`A`,..., `E`) into dummy variables or integer codes; similarly for `residence_area_type`.
- **Scaling:** Numeric features like `age`, `Income`, `premium`, and the computed ratios should be scaled (e.g. MinMax or StandardScaler) for algorithms that are sensitive to scale (logistic regression, neural nets). Tree-based models (XGBoost/RandomForest) need less scaling but it is still good practice to standardize numeric inputs.
- **Handling missing values:** After our initial cleaning, there are no missing values remaining. If we had chosen to keep the missing underwriting scores, we could impute (e.g. with the mean or a learned regression). In our approach we dropped them.
- **Class imbalance:** As noted, we will apply resampling or class-weighting *within* cross-validation. For example, using `class_weight='balanced'` in logistic regression, or applying SMOTE only on the training folds to avoid leakage. We must ensure any oversampling is done *after* splitting into train/test folds.
- **Feature importance check:** We may also use model-based feature selection (e.g. tree feature importances or L1 regularized logistic) to confirm our manual selections.

With this preparation, we proceed to modeling. Likely algorithms include: - **Logistic Regression:** A good baseline, with weighted classes. It offers interpretability via coefficients. - **XGBoost or LightGBM:** Gradient boosting handles heterogeneous data well; we can handle imbalance via scale_pos_weight or by balanced sampling. - **Neural Networks:** A simple feedforward net can be tried once features are scaled. - **Ensembles:** RandomForest, or ensemble of the above, possibly with imbalanced-focused variants. We will tune hyperparameters (via grid/random search) and evaluate using metrics like ROC-AUC, Precision-Recall, and the F1 score on the minority class.

Summary of Findings and Next Steps

- **Key predictors:** Late-payment history and percentage paid by cash/credit are the strongest signals (higher non-renewal associated with late payments and higher cash-credit fraction). Age, income, and channel also show effects.
- **Class imbalance:** The dataset is heavily skewed (≈94% renewals). We must use resampling or weighting to avoid bias toward the majority.

- **Prepared features:** We will use `total_late_counts` and related features, plus the numeric and categorical variables listed above. Numeric features will be normalized, and categorical features encoded.
- **Model candidates:** Logistic regression (with class weighting), gradient-boosted trees (XGBoost/LightGBM), and possibly a neural net. We will also experiment with imbalanced-ensemble methods if needed.

This completes the EDA and feature-engineering plan. The analysis above is thorough and suggests that the insurance renewal likelihood can be predicted using the chosen features, after careful handling of class imbalance and scaling. We next implement and validate the predictive models using these insights.

Sources: Mutual information properties [5]; imbalanced classification strategies and SMOTE/ADASYN methods [1] [2] [3].

---

[1] [2] SMOTE for Imbalanced Classification with Python - MachineLearningMastery.com
https://machinelearningmastery.com/smote-oversampling-for-imbalanced-classification/

[3] Oversampling and undersampling in data analysis - Wikipedia
https://en.wikipedia.org/wiki/Oversampling_and_undersampling_in_data_analysis

[4] SMOTE — Version 0.14.0
https://imbalanced-learn.org/stable/references/generated/imblearn.over_sampling.SMOTE.html

[5] mutual_info_classif — scikit-learn 1.7.2 documentation
https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.mutual_info_classif.html